

Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_4\feature\student_list> git branch
* feature/student-list
PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_4\feature\student_list> |
```

🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart `List`.

Research what a Dart `List` is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

Screenshot:

Code:

- Your student list

```

47
48 // Student list
49 final List<Student> students = [
50     const Student(
51         image: 'assets/121.jpg',
52         name: 'Keith Sins',
53         course: 'BS Information Technology',
54         yearLevel: '3rd Year',
55         age: 29,
56         hobby: 'Sleeping',
57         email: 'KeithSins@gmail.com',
58         favoriteSubject: 'Mobile Development',
59     ),
60     const Student(
61         image: 'assets/122.jpg',
62         name: 'Clyde lilicon',
63         course: 'BS Information Technology',
64         yearLevel: '3rd Year',
65         age: 17,
66         hobby: 'Basketball',
67         email: 'clyde.tibursho@gmail.com',
68         favoriteSubject: 'Algorithms',
69     ),
70     const Student(

```

-
- At least five student entries

```

70  const Student(
71      image: 'assets/123.jpg',
72      name: 'Antonio Badi',
73      course: 'BS Information Technology',
74      yearLevel: '3rd Year',
75      age: 21,
76      hobby: 'Reading',
77      email: 'antonioyadi@gmail.com',
78      favoriteSubject: 'Database Systems',
79  ),
80  const Student(
81      image: 'assets/124.jpg',
82      name: 'Carl gaye',
83      course: 'BS Information Technology',
84      yearLevel: '3rd Year',
85      age: 25,
86      hobby: 'Dancing',
87      email: 'carl.gaye@gmail.com',
88      favoriteSubject: 'Web Development',
89  ),
90  const Student(
91      image: 'assets/125.jpg',
92      name: 'Liam Badingon',
93      course: 'BS Information Technology',
94      yearLevel: '3rd Year',
95      age: 20,
96      hobby: 'Cycling',
97      email: 'liam.badingon@gmail.com',
98      favoriteSubject: 'Embedded Systems',
99  ),

```

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)

```
child: ConstrainedBox(
  constraints: const BoxConstraints(maxWidth: 760),
  child: ListView(
    padding: const EdgeInsets.all(16),
    children: [
      Padding(
        padding: const EdgeInsets.only(bottom: 14),
        child: StudentCard(student: students[0]),
      ),
      Padding(
        padding: const EdgeInsets.only(bottom: 14),
        child: StudentCard(student: students[1]),
      ),
      Padding(
        padding: const EdgeInsets.only(bottom: 14),
        child: StudentCard(student: students[2]),
      ),
      Padding(
        padding: const EdgeInsets.only(bottom: 14),
        child: StudentCard(student: students[3]),
      ),
      Padding(
        padding: const EdgeInsets.only(bottom: 14),
        child: StudentCard(student: students[4]),
      ),
    ],
  ),
),
```

Flag 3 — Discover **ListView**

Objective: Make your student directory scrollable using **ListView**.

Research Flutter's **ListView**.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

Test

The user should be able to:

- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:

- `ListView` in your code

```
? const _EmptyStudentState()
: Center(
  child: ConstrainedBox(
    constraints: const BoxConstraints(maxWidth: 760),
    child: ListView(
      padding: const EdgeInsets.all(16),
      children: [
        Padding(
          padding: const EdgeInsets.only(bottom: 14),
          child: StudentCard(student: students[0]),
        ),
        Padding(
          padding: const EdgeInsets.only(bottom: 14),
          child: StudentCard(student: students[1]),
        ),
      ],
    ),
  ),
)
```

- Top of the application
- Bottom portion showing later students

student_list

Student Directory

6 Students



Keith Sins

BS Information Technology

3rd Year

Age

29

Hobby

Sleeping

Email

KeithSins@gmail.com

Favorite Subject

Mobile Development



Clyde lilicon

BS Information Technology

3rd Year

Age

17

Hobby

Basketball

Email

clyde.tibursho@gmail.com

Favorite Subject

Algorithms



Antonio Badi

student_list

Student Directory

6 Students



Antonio Badi

BS Information Technology

3rd Year

Age

21

Hobby

Reading

Email

antoniobadi@gmail.com

Favorite Subject

Database Systems



Carl gaye

BS Information Technology

3rd Year

Age

25

Hobby

Dancing

Email

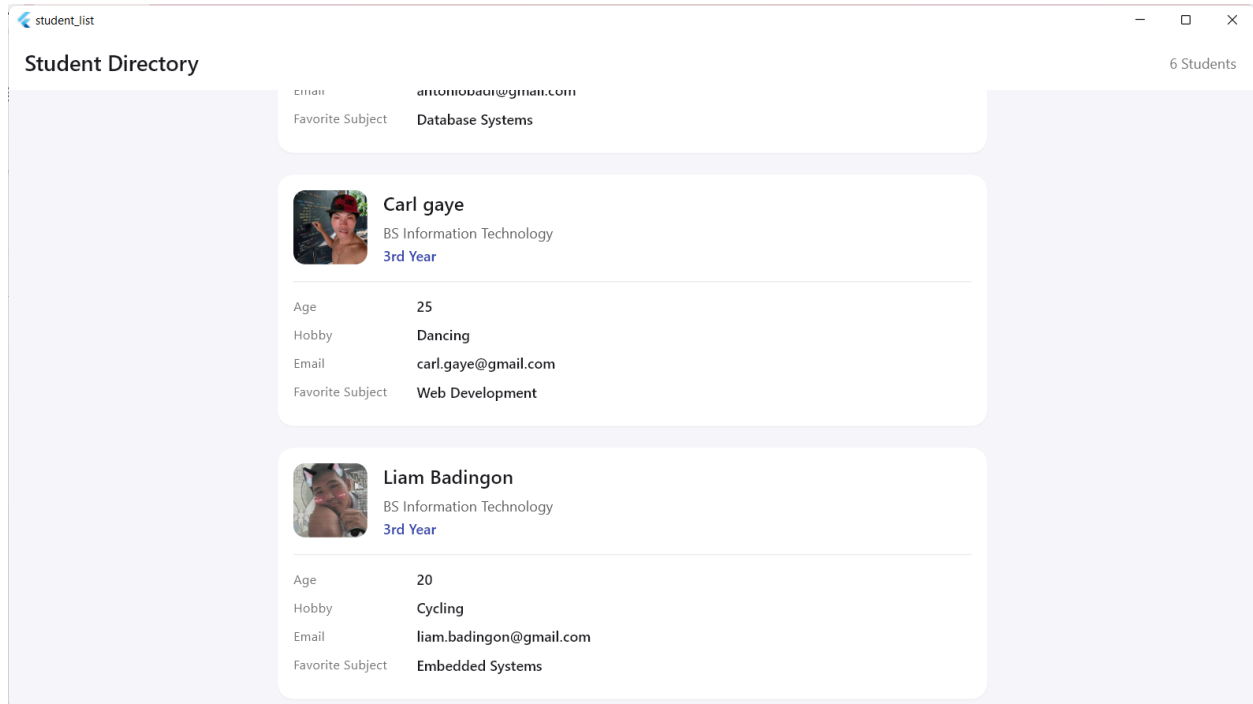
carl.gaye@gmail.com

Favorite Subject

Web Development



Liam Badinger



🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

Test

Add one new student to your list.

Without creating another card manually, your application should display the new student automatically.

Screenshot:

- `ListView.builder` in your code

```

Widget build(BuildContext context) {
  ), // Padding
  Padding(
    padding: const EdgeInsets.only(bottom: 14),
    child: StudentCard(student: students[1]),
  ), // Padding
  Padding(
    padding: const EdgeInsets.only(bottom: 14),
    child: StudentCard(student: students[2]),
  ), // Padding
  Padding(
    padding: const EdgeInsets.only(bottom: 14),
    child: StudentCard(student: students[3]),
  ), // Padding
  Padding(
    padding: const EdgeInsets.only(bottom: 14),
    child: StudentCard(student: students[4]),
  ), // Padding
  Padding(
    padding: const EdgeInsets.only(bottom: 14),
    child: StudentCard(student: students[5]),
  ), // Padding
  ],
), // ListView
), // ConstrainedBox
) // Center

```

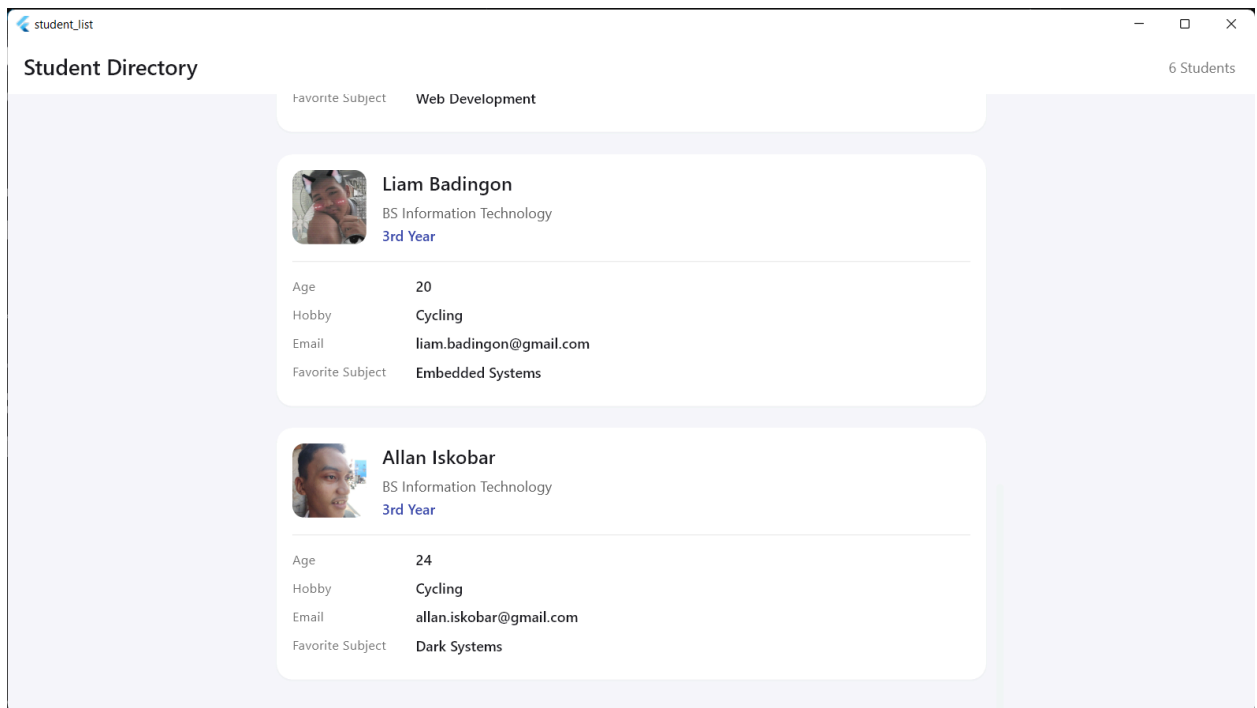
- Student list

```

98     favoriteSubject: 'Embedded Systems',
99   ),
100   const Stu Type: String
101     image: 'assets/parbs.jpg',
102     name: 'Allan Iskobar',
103     course: 'BS Information Technology',
104     yearLevel: '3rd Year',
105     age: 24,
106     hobby: 'Cycling',
107     email: 'allan.iskobar@gmail.com',
108     favoriteSubject: 'Dark Systems',
109   ),
110 ];
111
112 class StudentDirectoryPage extends StatelessWidget {
113   const StudentDirectoryPage({super.key});
114

```

- Newly generated student appearing in the app
-



🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

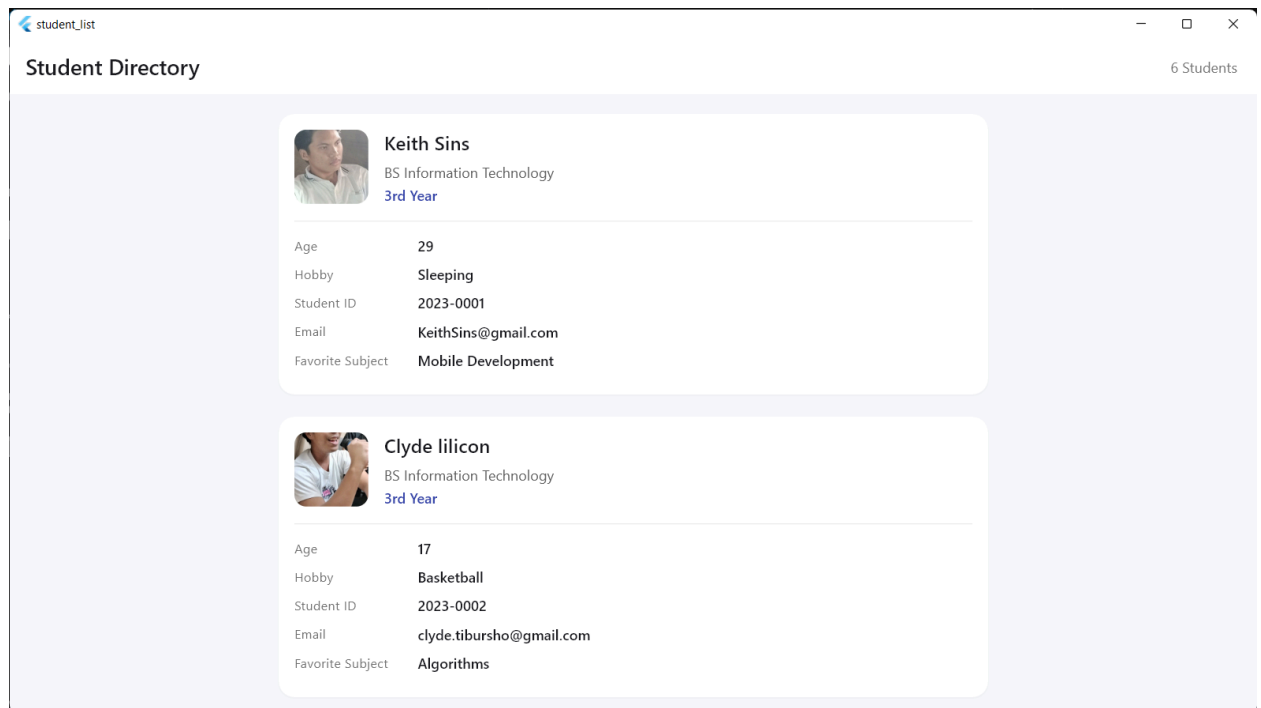
Examples:

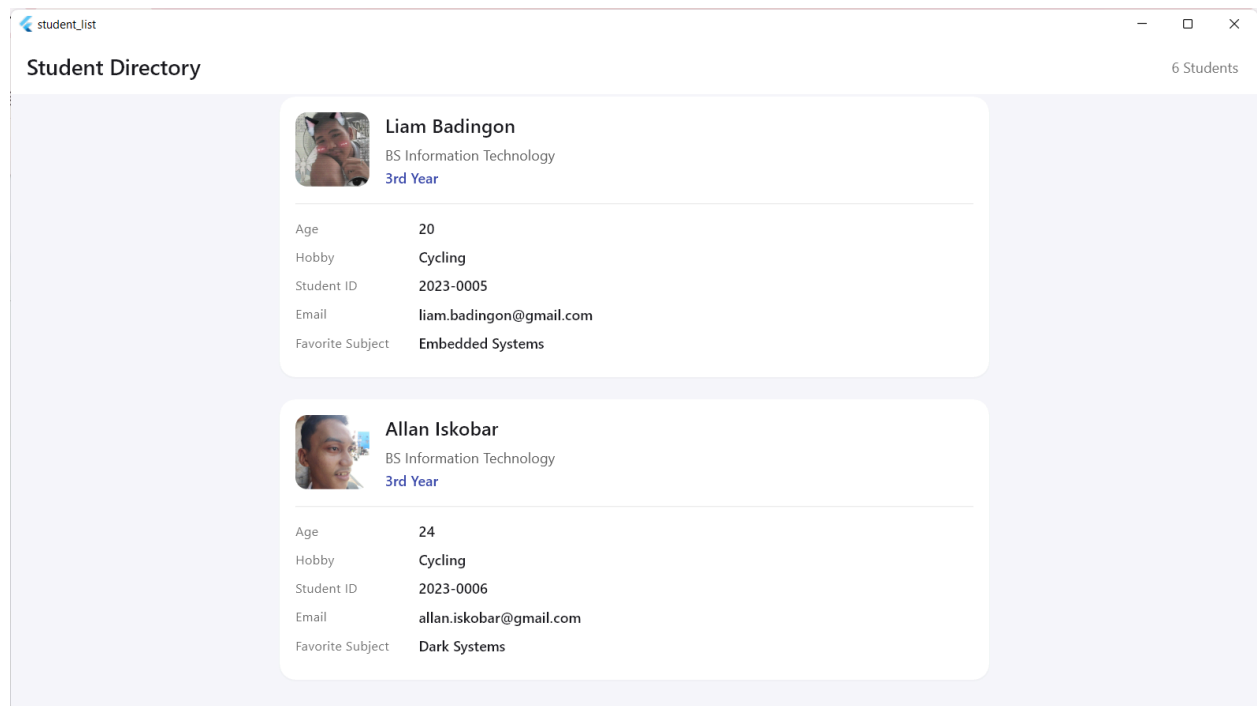
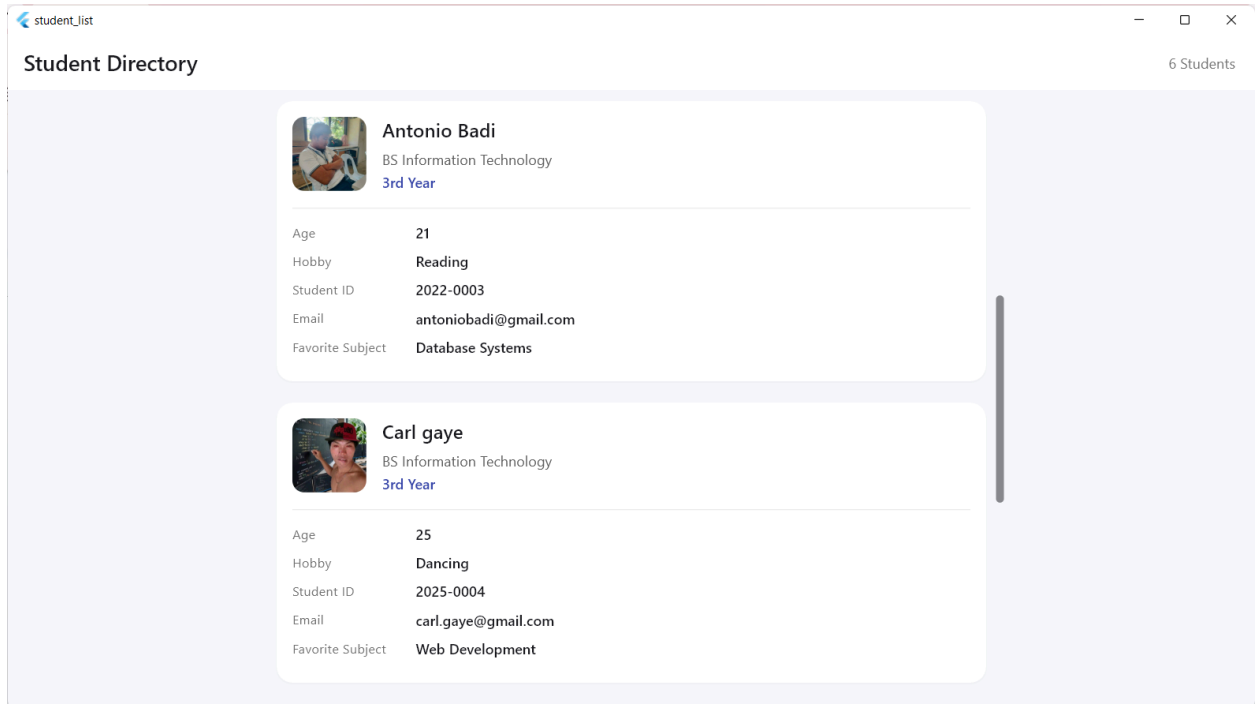
- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

Screenshot:

- Updated student data
- Student Card displaying the additional fields





🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty **List** gracefully.

Imagine your app just connected to a database.

The database returns:

```
[]
```

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

- "No students found."
- "Student list is empty."

Test

- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

Screenshot:

- Empty student list

```
334
335 class _EmptyStudentState extends StatelessWidget {
336   const _EmptyStudentState();
337
338   @override
339   Widget build(BuildContext context) {
340     return Center(
341       child: Text(
342         'No students found.',
343         style: TextStyle(fontSize: 16, color: Colors.grey.shade600),
344       ), // Text
345     ); // Center
346   }
347 }
348
```

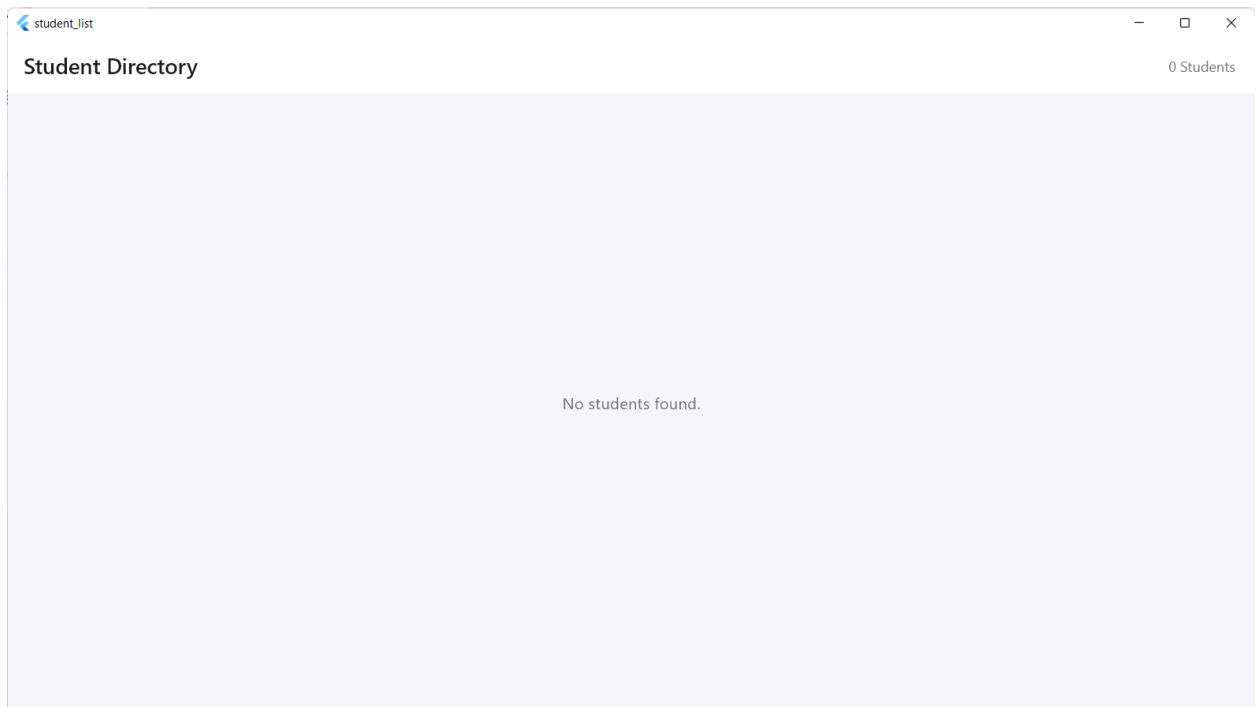
- Empty-state UI

```

9
10 // Student list
11 final List<Student> students = [];
12
13 class StudentDirectoryPage extends StatelessWidget {
14   const StudentDirectoryPage({super.key});
15

```

- Application still functioning correctly



🚩 Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a `List`.

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

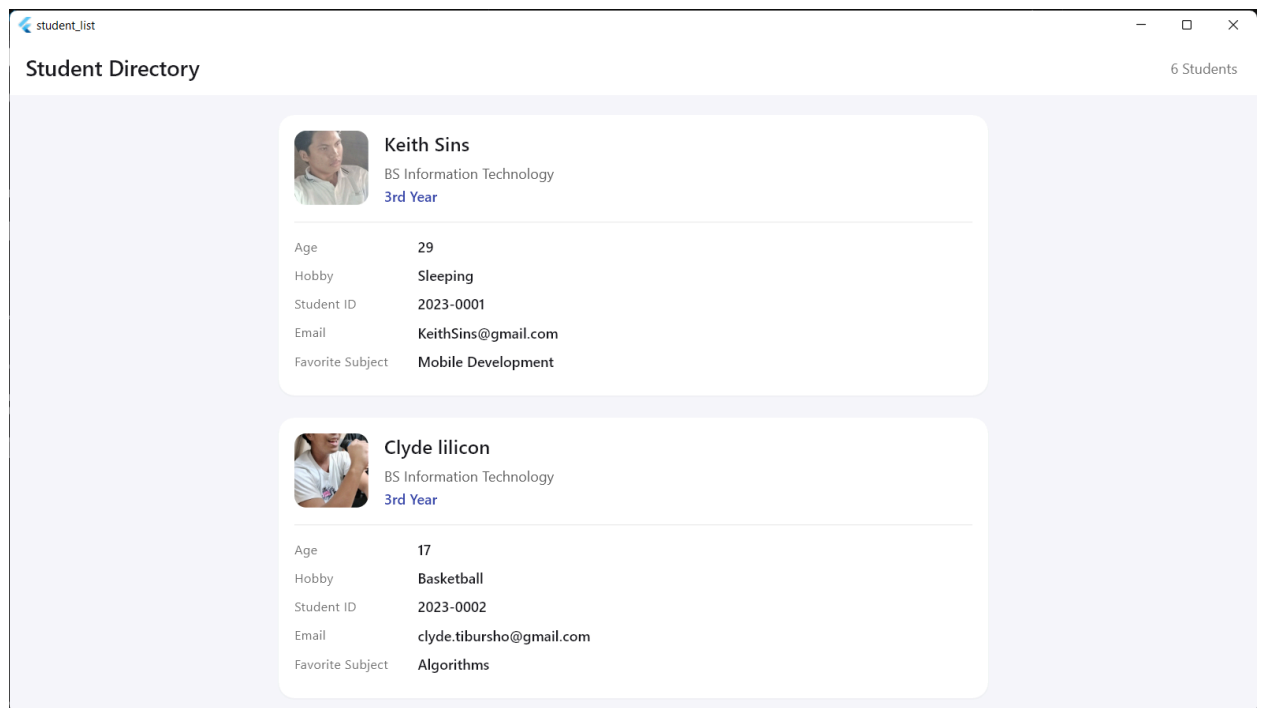
After sorting:

- Aaron
- John
- Maria
- Sophia

Your UI should immediately reflect the new order.

Screenshot:

- Student list before or after sorting



- Application displaying the sorted order

student_list

Student Directory

6 Students



Allan Iskobar

BS Information Technology

3rd Year

Age

24

Hobby

Cycling

Student ID

2023-0006

Email

allan.iskobar@gmail.com

Favorite Subject

Dark Systems



Antonio Badi

BS Information Technology

3rd Year

Age

21

Hobby

Reading

Student ID

2022-0003

Email

antonibadi@gmail.com

Favorite Subject

Database Systems

student_list

Student Directory

6 Students



Carl gaye

BS Information Technology

3rd Year

Age

25

Hobby

Dancing

Student ID

2025-0004

Email

carl.gaye@gmail.com

Favorite Subject

Web Development



Clyde lilicon

BS Information Technology

3rd Year

Age

17

Hobby

Basketball

Student ID

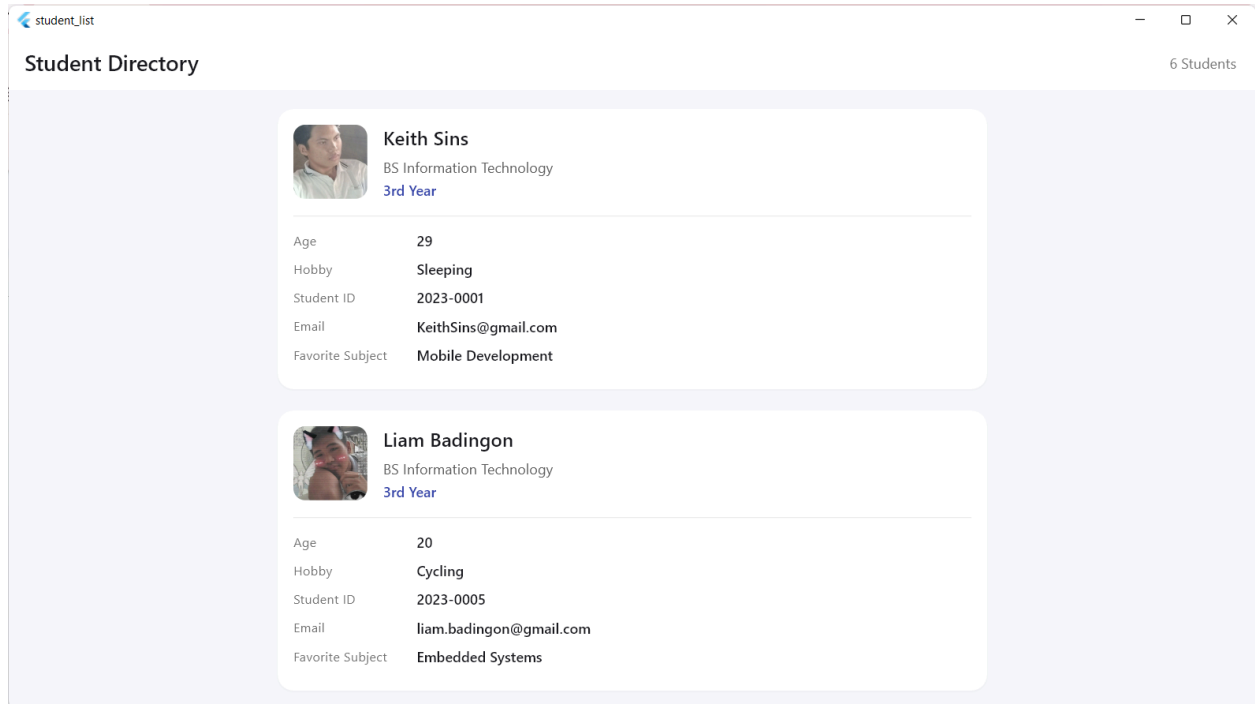
2023-0002

Email

clyde.tibursho@gmail.com

Favorite Subject

Algorithms



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/student-list`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

MINGW64:/c/Users/Ninja Turtle/Documents/binsoy/CTF_4/feature/student_list

```
warning: in the working copy of 'linux/flutter/generated_plugin_registrant.h', LF
will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'linux/flutter/generated_plugins.cmake', LF will
be replaced by CRLF the next time Git touches it
warning: in the working copy of 'macos/Flutter/GeneratedPluginRegistrant.swift',
LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'pubspec.lock', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'windows/flutter/generated_plugin_registrant.cc'
, LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'windows/flutter/generated_plugin_registrant.h',
LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'windows/flutter/generated_plugins.cmake', LF wi
ll be replaced by CRLF the next time Git touches it
```

```
Ninja Turtle@DESKTOP-SGCJ92Q MINGW64 ~/Documents/binsoy/CTF_4/feature/student_li
st (feature/student-list)
```

```
$ git commit -m "feat: complete student_list flutter project"
[feature/student-list 6cc15c9] feat: complete student_list flutter project
143 files changed, 5178 insertions(+), 128 deletions(-)
create mode 100644 .gitignore
create mode 100644 .metadata
create mode 100644 .vs/CMakeWorkspaceSettings.json
create mode 100644 .vs/ProjectSettings.json
```